

Quantum Neural Networks (QNNs) — Detailed Notes (Session 14)

Course: CS490/5590 — Quantum Computing Applications in Data Science, AI, & DL

Instructor: Luke Miller

Goal: A stand-alone reference for building, training, and evaluating QNNs on NISQ hardware/sim.

Road-map

- Recap: feature selection → fewer qubits, better trainability
- What is a QNN? (PQC = model)
- Circuit learning: encoding, ansatz, readout/loss
- Variational quantum classifiers (VQC) & regressors (VQR)
- Training: gradients, optimizers, regularization
- Qiskit ML workflows (high-level & low-level)
- QNNs vs classical NNs
- Applications, pitfalls, mini-exercises

0) Recap — why feature selection first?

- **Qubits $\approx$ features encoded.** Fewer features $\rightarrow$ **shallower** circuits, **less noise**, better gradients.
- Use classical filters or quantum kernel/QMI (from Session 11) to pre-select features.
- **Takeaway:** Do feature selection before designing the QNN.

1) What is a QNN?

A **parameterized quantum circuit (PQC)** that maps classical input x to a state and outputs expectations via measurement.

$$|\psi(x; \theta)\rangle = U_{\text{ansatz}}(\theta) U_{\phi}(x) |0\rangle^{\otimes n}$$

- $U_{\phi}(x)$: feature map (e.g., $R_Z(\alpha x_i)$, ZZ entangling terms).
- $U_{\text{ansatz}}(\theta)$: trainable layers (e.g., TwoLocal with RY/CX, repeated p times).
- **Outputs:** single $\langle P \rangle$ (binary) or weighted sum $\langle \sum_i w_i P_i \rangle$ (regression).
- **Why NISQ-friendly?** Shallow variational circuits + classical optimization.

2) Circuit learning — the pieces

2.1 Data encoding (feature map)

- **Angle encoding:** $R_Z(\alpha x_i), R_Y(\alpha x_i)$ per qubit; optional ZZ entangling $e^{-i\beta x_i x_j Z_i Z_j}$.
- **Data re-uploading:** interleave $U_\phi(x)$ between trainable layers to boost expressivity without more qubits.
- **Normalization:** scale inputs to $[-\pi, \pi]$ (or similar) to avoid saturation.

2.2 Ansatz design

- **Hardware-efficient:** RY/RZ + CX aligned to device connectivity (shallow, trainable).
- **Problem-inspired:** encode symmetries/conservation laws (fewer params, better bias).
- **Depth/width trade-off:** more layers $\uparrow$ expressivity **but** $\uparrow$ barren-plateau risk & noise.

2.3 Readout & loss

- **Binary classification:** $\hat{y} = (1 - \langle Z_0 \rangle)/2 \in [0, 1]$; loss: BCE or hinge.
- **Regression:** $f(x) = \sum_i w_i \langle Z_i \rangle$; loss: MSE (rescale if target $\notin [-1, 1]$).
- **Multiclass:** use k readout qubits (one-vs-rest) or measure a vector and apply softmax classically.

2) Circuit learning — the pieces

2.1 Data encoding (feature map)

- **Angle encoding:** $R_Z(\alpha x_i)$, $R_Y(\alpha x_i)$ per qubit; optional ZZ entangling $e^{-i\beta x_i x_j Z_i Z_j}$.
- **Data re-uploading:** interleave $U_\phi(x)$ between trainable layers to boost expressivity without more qubits.
- **Normalization:** scale inputs to $[-\pi, \pi]$ (or similar) to avoid saturation.

2.2 Ansatz design

- **Hardware-efficient:** RY/RZ + CX aligned to device connectivity (shallow, trainable).
- **Problem-inspired:** encode symmetries/conservation laws (fewer params, better bias).
- **Depth/width trade-off:** more layers $\uparrow$ expressivity **but** $\uparrow$ barren-plateau risk & noise.

2.3 Readout & loss

- **Binary classification:** $\hat{y} = (1 - \langle Z_0 \rangle)/2 \in [0, 1]$; loss: BCE or hinge.
- **Regression:** $f(x) = \sum_i w_i \langle Z_i \rangle$; loss: MSE (rescale if target $\notin [-1, 1]$).
- **Multiclass:** use k readout qubits (one-vs-rest) or measure a vector and apply softmax classically.

3) Variational Quantum Classifiers (VQC)

Pipeline:

1. Encode x via $U_\phi(x)$
2. Apply $U(\theta)$
3. Measure $\langle Z_0 \rangle$ (or a vector)
4. Compute loss vs label y , update θ

Design tips:

- Start with **RY+CX** (or RY–RZ + CX), **1–3 reps**.
- Entangle **per coupling map**; avoid all-to-all on sparse devices.
- Regularize: **L2 on θ** , early stopping, shot-aware validation.

4) Variational Quantum Regressors (VQR)

- Same structure; target $t \in \mathbb{R}$.
- Scale/squash outputs if needed ($[-1, 1]$ range of $\langle Z \rangle$).
- Shots impose a precision floor; average more shots or apply readout calibration.

5) Training QNNs

5.1 Gradients — parameter-shift

For a gate $e^{-i\theta_k P/2}$ with Pauli P ,

$$\frac{\partial}{\partial \theta_k} \langle H \rangle = \frac{1}{2} \left(\langle H \rangle_{\theta_k + \frac{\pi}{2}} - \langle H \rangle_{\theta_k - \frac{\pi}{2}} \right).$$

- Exact for many native rotations. Cost: **2 evaluations per parameter** (times shots).

5.2 Optimizers

- **Gradient-based:** Adam/SGD (needs parameter-shift).
- **Gradient-free:** SPSA (2 evals/step, noise-robust), COBYLA/Nelder–Mead (few params).
- **Scheduling:** cosine/step LR; mini-batches for larger datasets.

5.3 Trainability & regularization

- **Barren plateaus:** keep shallow, use local costs, layer-wise growth, small-init θ , feature selection.
- **Noise:** readout calibration, dynamical decoupling, zero-noise extrapolation; moderate reps.

5) Training QNNs

5.1 Gradients — parameter-shift

For a gate $e^{-i\theta_k P/2}$ with Pauli P ,

$$\frac{\partial}{\partial \theta_k} \langle H \rangle = \frac{1}{2} \left(\langle H \rangle_{\theta_k + \frac{\pi}{2}} - \langle H \rangle_{\theta_k - \frac{\pi}{2}} \right).$$

- Exact for many native rotations. Cost: **2 evaluations per parameter** (times shots).

5.2 Optimizers

- **Gradient-based:** Adam/SGD (needs parameter-shift).
- **Gradient-free:** SPSA (2 evals/step, noise-robust), COBYLA/Nelder–Mead (few params).
- **Scheduling:** cosine/step LR; mini-batches for larger datasets.

5.3 Trainability & regularization

- **Barren plateaus:** keep shallow, use local costs, layer-wise growth, small-init θ , feature selection.
- **Noise:** readout calibration, dynamical decoupling, zero-noise extrapolation; moderate reps.

Example

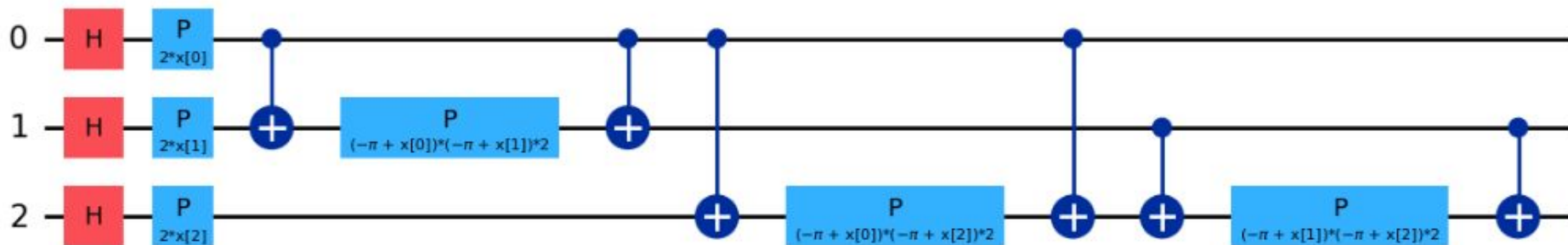
```
1 import qiskit
2 from qiskit import QuantumCircuit
3 from qiskit.quantum_info import SparsePauliOp
4 from qiskit_aer import AerSimulator
5 from scipy.optimize import minimize
6 from qiskit.circuit.library import zz_feature_map, n_local
7 from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
8 from qiskit_ibm_runtime import EstimatorV2 as Estimator
9 import numpy as np
10 from sklearn import datasets
```

```
12 # Load and preprocess data |
13 iris = datasets.load_iris()
14 X = iris.data[:, :3] # 3 features
15 X = (X - X.min(0)) / (X.max(0) - X.min(0) + 1e-12) # Normalize to [0,1]
16 y = iris.target # Classes {0,1,2}
17 split = 4 * (len(X) // 5)
18 X_train, y_train, X_test, y_test = X[:split], y[:split], X[split:], y[split:]
```

```
20 # Define quantum circuit
21 full_circuit = QuantumCircuit(3)
22 feature_map = zz_feature_map(feature_dimension=3, reps=1)
23 full_circuit = full_circuit.compose(feature_map)
24 full_circuit.barrier()
25 ansatz = n_local(num_qubits=3, rotation_blocks='ry', entanglement_blocks='cx', reps=2)
26 full_circuit = full_circuit.compose(ansatz)
```

A quick example of what our circuits look like

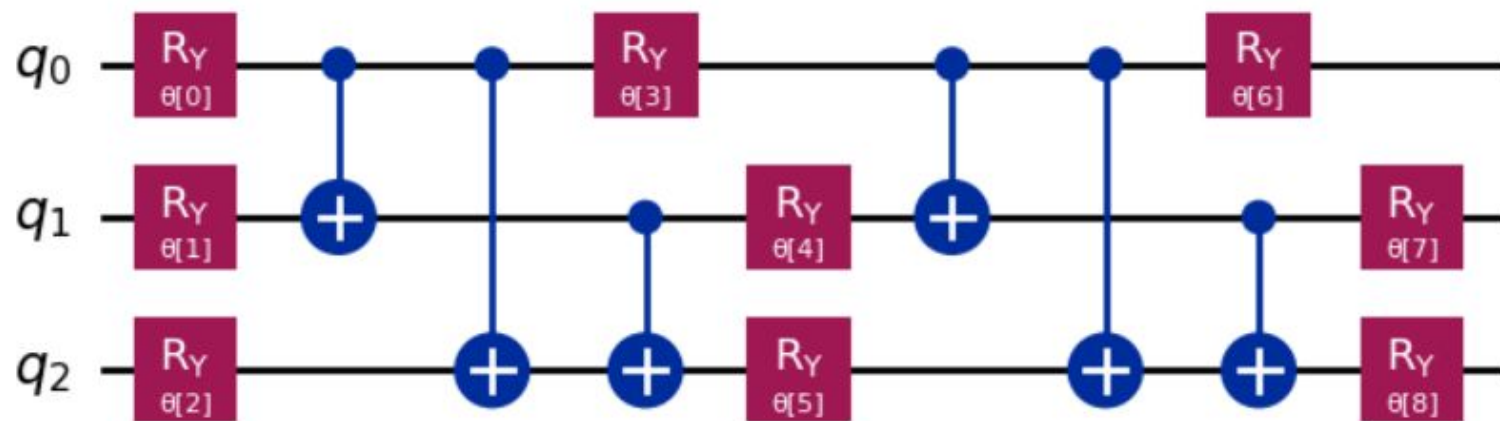
```
import pylatexenc
import qiskit
from qiskit import QuantumCircuit
from qiskit.circuit.library import zz_feature_map, n_local
QC = QuantumCircuit(3)
feature_map = zz_feature_map(feature_dimension=3, reps=1)
feature_map.draw('mpl')
```



```

1 import pylatexenc
2 import qiskit
3 from qiskit import QuantumCircuit
4 from qiskit.circuit.library import zz_feature_map, n_local
5 QC = QuantumCircuit(3)
6 feature_map = zz_feature_map(feature_dimension=3, reps=1)
7 # feature_map.draw('mpl')
8 ansatz = n_local(num_qubits=3, rotation_blocks='ry', entanglement_blocks='cx', reps=2)
9 ansatz.draw('mpl')

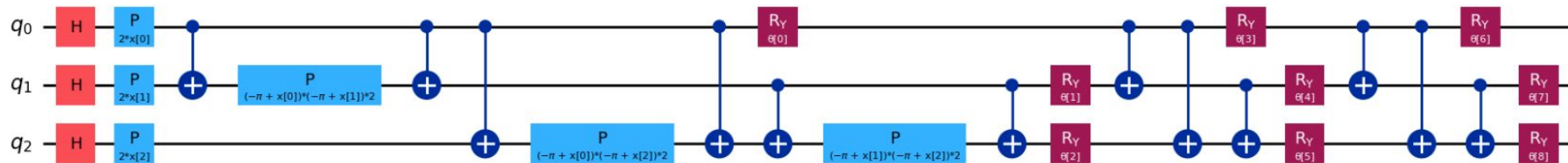
```



```

import pylatexenc
import qiskit
from qiskit import QuantumCircuit
from qiskit.circuit.library import zz_feature_map, n_local
QC = QuantumCircuit(3)
feature_map = zz_feature_map(feature_dimension=3, reps=1)
# feature_map.draw('mpl')
ansatz = n_local(num_qubits=3, rotation_blocks='ry', entanglement_blocks='cx', reps=2)
# ansatz.draw('mpl')
QC = QC.compose(feature_map).compose(ansatz)
QC.draw('mpl', fold=30)

```



...back to code

```
8 # Observable (sum of expectations across terms)
9 observable = SparsePauliOp(['XXI', 'IXX', 'XIX', 'XZI', 'IXZ', 'ZIX'])
0
1 # Backend and transpilation
2 backend = AerSimulator()
3 pm = generate_preset_pass_manager(backend=backend, optimization_level=1)
4 transpiled_circuit = pm.run(full_circuit)
5 transpiled_observable = observable # No layout change needed for simulator
6
7 # Estimator
8 estimator = Estimator(mode=backend)
```



```
40 # Forward function (batched via single PUB)
41 def forward(circuit, input_params, weight_params, estimator, observable):
42     if input_params.ndim == 1:
43         input_params = input_params.reshape(1, -1)
44
45     batch_size = input_params.shape[0]
46     weights = np.broadcast_to(weight_params, (batch_size, len(weight_params)))
47     params = np.concatenate((input_params, weights), axis=1)
48
49     # PUB: single circuit + observable + list of all param sets for batching
50     pub = [(circuit, observable, params)]
51
52     # Run estimator
53     job = estimator.run(pub)
54     expectation_values = job.result()[0].data.evs # Array of scalars (one per param set)
55
56     return expectation_values
```

```
59 def mse_loss(predict, target): return ((predict - target) ** 2).mean()
60
61 # Objective for optimization
62 objective_func_vals = []
63 iter = 0
64 def mse_loss_weights(weight_params):
65     global iter
66     predictions = forward(
67         transpiled_circuit, input_params, weight_params, estimator, transpiled_observable
68     )
69     cost = mse_loss(predictions, target)
70     objective_func_vals.append(cost)
71     if iter % 50 == 0:
72         print(f"Iter: {iter}, loss: {cost}")
73     iter += 1
```

```
76 # Training loop
77 batch_size = 30
78 num_epochs = 5
79 num_samples = len(X_train)
80
81 # Random initial weights
82 np.random.seed(42)
83 weight_params = np.random.rand(len(ansatz.parameters)) * 2 * np.pi
```

```
85 for epoch in range(num_epochs):
86     print(f"Epoch: {epoch}")
87     for i in range((num_samples - 1) // batch_size + 1):
88         print(f"    Batch: {i}")
89         start_i = i * batch_size
90         end_i = min(start_i + batch_size, num_samples) # Avoid index overflow
91         train_batch = np.array(X_train[start_i:end_i])
92         labels_batch = np.array(y_train[start_i:end_i])
93         input_params = train_batch
94         target = labels_batch
95         iter = 0 # Reset iter per optimization
96         res = minimize(
97             mse_loss_weights, weight_params, method="COBYLA", options={"maxiter": 100}
98         )
99         weight_params = res["x"] # Update weights after optimization
100     print(f"Epoch {epoch} complete. Final batch loss: {objective_func_vals[-1]:.4f}")
```